115-1 (Fall 2026) Semester

Reinforcement Learning

Assignment #3

TA: Chon-Ho Chang(張冲和)

Department of Electrical Engineering
National Taiwan University

Outline

- What to learn from assignment #3 ?
- Stable baseline 3 & Gymnasium
- Env 1 - 2048
- Env 2 - Astro Party
- Code structure
- Grading
- Report
- Policy
- Submission
- Contact

What to learn from assignment #3?

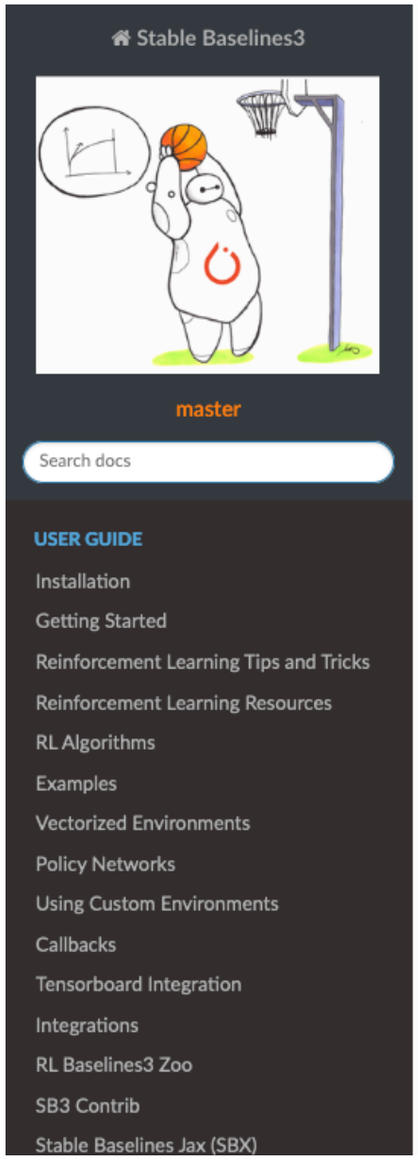
What to learn from assignment #3 ?

- Part 1 :
  - How to do Q-Learning with deep learning?
  - What common fault does Q-Learning induce? How to solve it?
- Part 2 :
  - How to do RL in continuous action space?
  - Tips and tricks when RL from scratch is too hard.

Stable Baselines 3 & Gymnasium

Stable Baselines 3

- Stable Baselines is a package that provides training APIs for RL training.
- Useful for training Gym environment on basic RL algorithms.
- [Documentation]



Supported Algorithms

- A2C (Advantage Actor Critic)
- DDPG (Deep Deterministic Policy Gradient)
- DQN (Deep Q Network)
- PPO (Proximal Policy Optimization)
- SAC (Soft Actor Critic)
- TD3 (Twin Delayed DDPG)

- Note that some algorithms don’t support every action and observation space type.
- Make sure that the algorithm you pick fits the environment for this assignment.
- Read RL Algorithms and the documentation of each algorithm for more detail.

Policy Networks

- Each algorithm (PPO, DQN, A2C, etc.) has a built-in policy class.
  - 64 units (per layer) for PPO/A2C/DQN
  - 256 units for SAC

- Types of Policies
  - MlpPolicy: For vector inputs (e.g., numbers)
  - CnnPolicy: For image inputs
  - MultiInputPolicy: For environments with mixed inputs (e.g., images + vectors)

- Customization:
  - Policy network:

```
policy_kwargs = dict(net_arch=[dict(pi=[128, 128], vf=[64, 64])])
```

  - Features extractor:

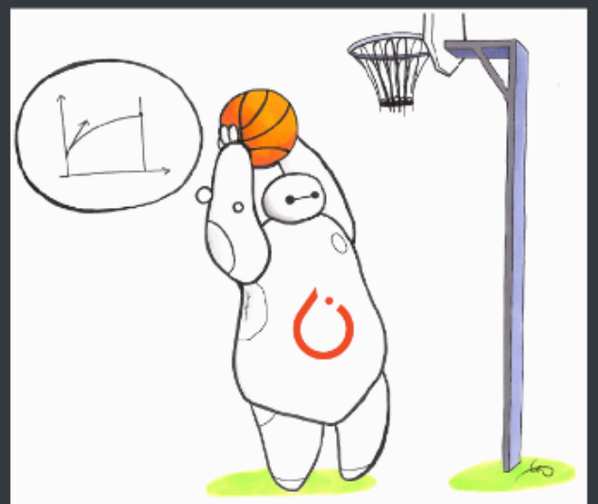
```
from stable_baselines3.common.torch_layers import BaseFeaturesExtractor

class Custom2048CNN(BaseFeaturesExtractor):
    ...
```

Tutorials

- Stable Baselines 3 Examples

Stable Baselines3



master

Search docs

USER GUIDE

Installation

Getting Started

Reinforcement Learning Tips and Tricks

Reinforcement Learning Resources

RL Algorithms

Examples

Try it online with Colab Notebooks!

Examples / Edit on GitHub

Examples

Note

These examples are only to demonstrate the use of the library and its functions, and the trained agents may not solve the environments. Optimized hyperparameters can be found in the RL Zoo repository.

```python
import gymnasium as gym

from stable_baselines3 import DQN
from stable_baselines3.common.evaluation import evaluate_policy

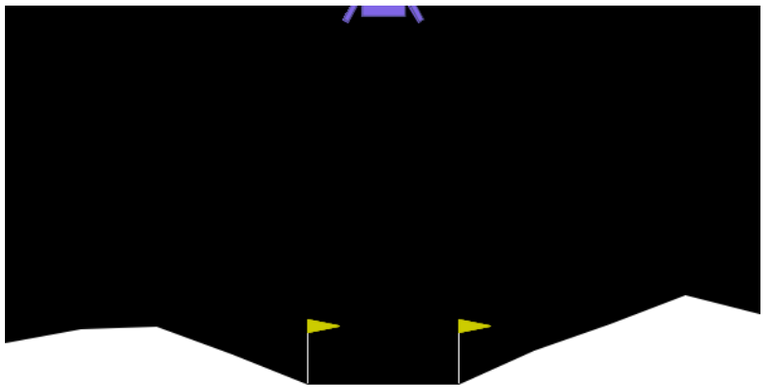

# Create environment
env = gym.make("LunarLander-v2", render_mode="rgb_array")

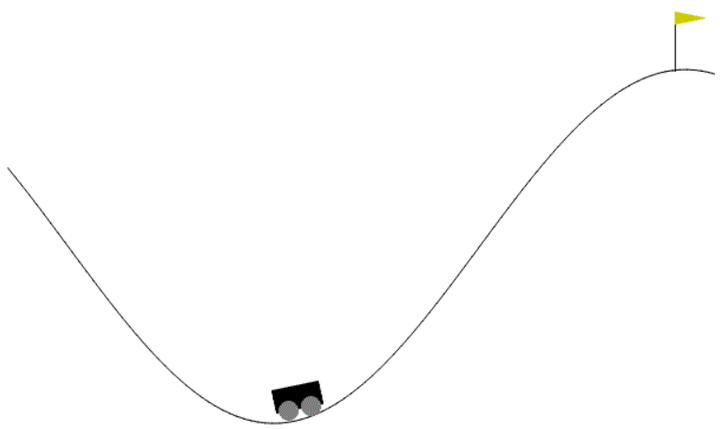
# Instantiate the agent
model = DQN("MlpPolicy", env, verbose=1)
# Train the agent and display a progress bar
model.learn(total_timesteps=int(2e5), progress_bar=True)
# Save the agent
model.save("dqn_lunar")
del model  # delete trained model to demonstrate loading

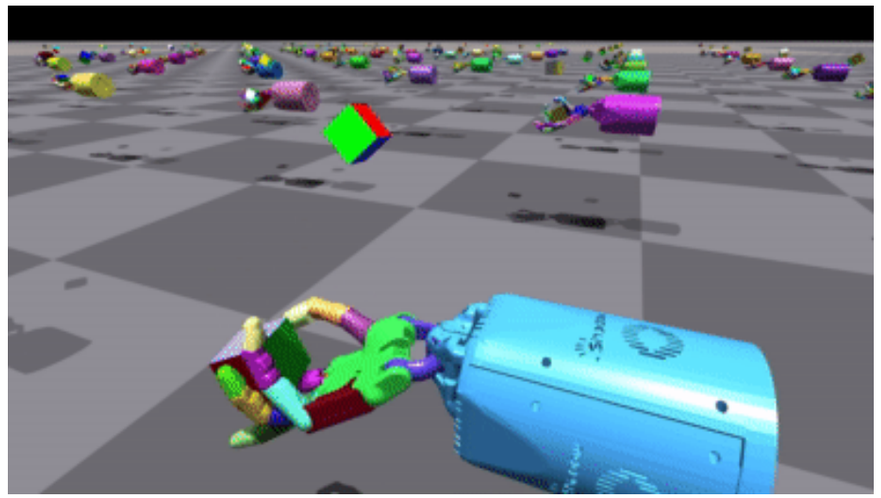
# Load the trained agent
```

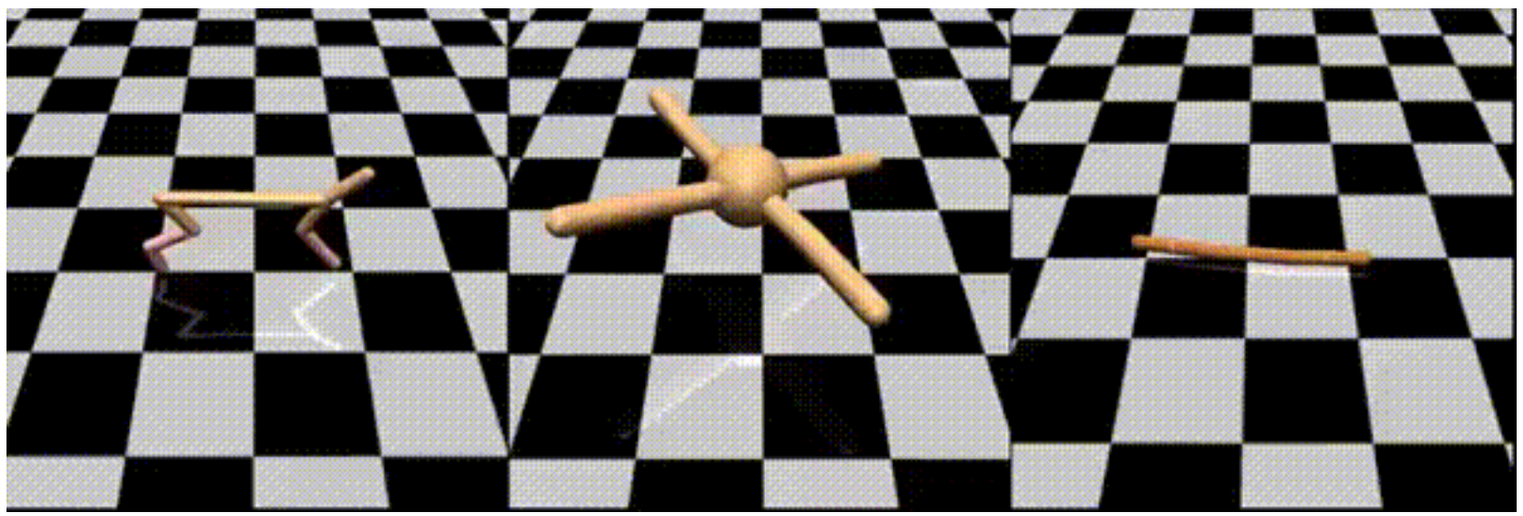
Environment Packages

- There are many open-source environment packages for RL training.
- Examples: “OpenAI Gym”, “Deepmind Control”, “Nvidia Isaac Gym”, etc.
- For this assignment, we will mainly focus on “OpenAI Gym / Gymnasium” for its compatibility with “Stable Baselines 3”.



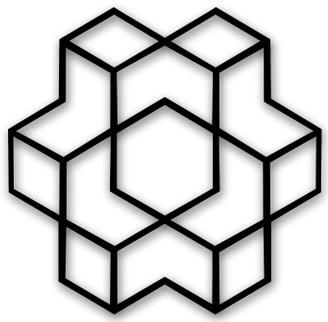






OpenAI Gym / Gymnasium

- Gym is an open source package developed by OpenAI to provide a standard API to communicate between learning algorithms and environments.
- The Farama Foundation announced the release of Gymnasium by October 2022 and took over the future maintenance of OpenAI Gym. (Announcing The Farama Foundation)
- For this assignment, we will be using Gymnasium.
- OpenAI Gym / Gymnasium





Environment Methods and Tips

- Methods: (For Gymnasium or Gym version after v0.26.0)
  - step(action): Interacts with the environment. Returns obs, rewards, termination, truncation, info.
    - "True" if episode ends because it reaches max episode length.
    - "True" if the episode ends due to termination conditions. (ex: Agent made illegal moves or finished a task)
  - reset(): Resets the environment. Returns obs, info.
    - A dictionary of information that might be useful. (ex: {"success": False, "score": 42})

Custom Environments

- Step 1: Create Your Environment Class
  - Save it in “my_module/my_env/“

```python
import gymnasium as gym
from gymnasium import spaces

class MyCustomEnv(gym.Env):
    def __init__(self):
        super().__init__()
        ...
    def reset(self):
        ...
    def step(self, action):
        ...
```

- Step 2: Register the Environment
  - Register in your training script

```python
from gymnasium.envs.registration import register

register(
    id="MyCustomEnv-v0",
    entry_point="my_module.my_env:MyCustomEnv",
)
```

- Step 3: Make the Environment
  - Make the env before training or eval

```python
import gymnasium as gym

env = gym.make("MyCustomEnv-v0")
```

Vectorized Environments

- Vectorized Environments are used to stack multiple independent environments into a single environment.
- Training the agent on a parallel environment helps speed up the training process.
- You can use VecEnv for training and still use the original Env for evaluation.
- Example: SubprocVecEnv can create separate envs that agents interact with in parallel.
- Note: VecEnv wraps the environment with Gym 0.21 API logic, so:
  - Output of VecEnv.step() would be obs, reward, done, info instead.
  - Output of VecEnv.reset() would be obs only.

    (See VecEnv API vs Gym API)

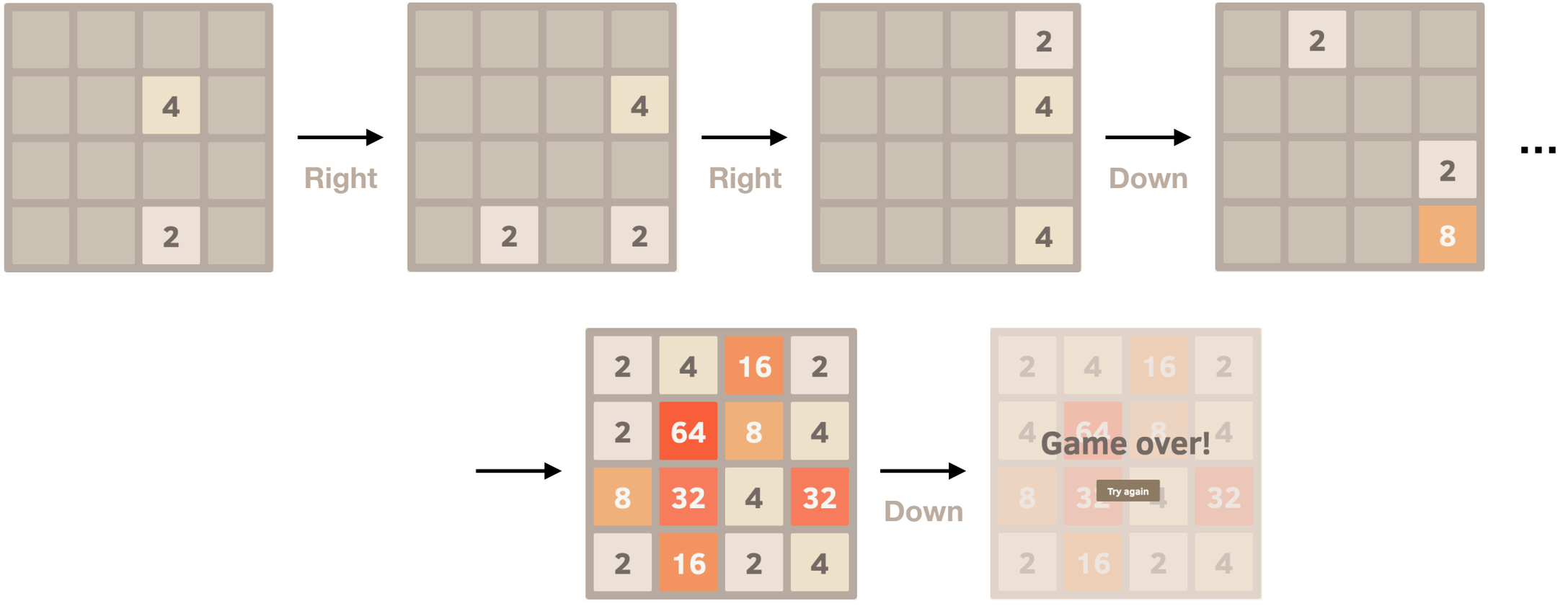
Notes

- There was a major update for the Gym API after version 0.26.0
- If you're using environments from older versions of Gym, you might want to watch out for compatibility issues.
- Release notes for v0.26.0

Env 1 - 2048

Env 1 - 2048 - Rules

- Use ↑, ↓, → and ← to move number tiles on the board.
- When the same number collide, they will merge into a tile with the total value of two tiles.
- You will also get scores for each tiles combined.
- A new tile (with value 2 or 4) will generate in a random empty space after each move.
- The game ends when there is no legal move left.
- Play 2048



Env 1 - 2048 - Termination

- For this environment, there will be 2 scenarios causing the step function to return True for termination.
  1. Termination for game end:
     - The episode terminates when there is no legal move left.
  2. Termination for illegal moves:
     - To avoid infinite loops, the environment will consider action that causes no state change an “illegal move”.
     - In evaluation, the episode terminates when the agent executes an illegal move.
     - You can modify the training code to let agent explore other actions even after illegal moves.

Env 1 - 2048 - State and action space

$n = 2^k$

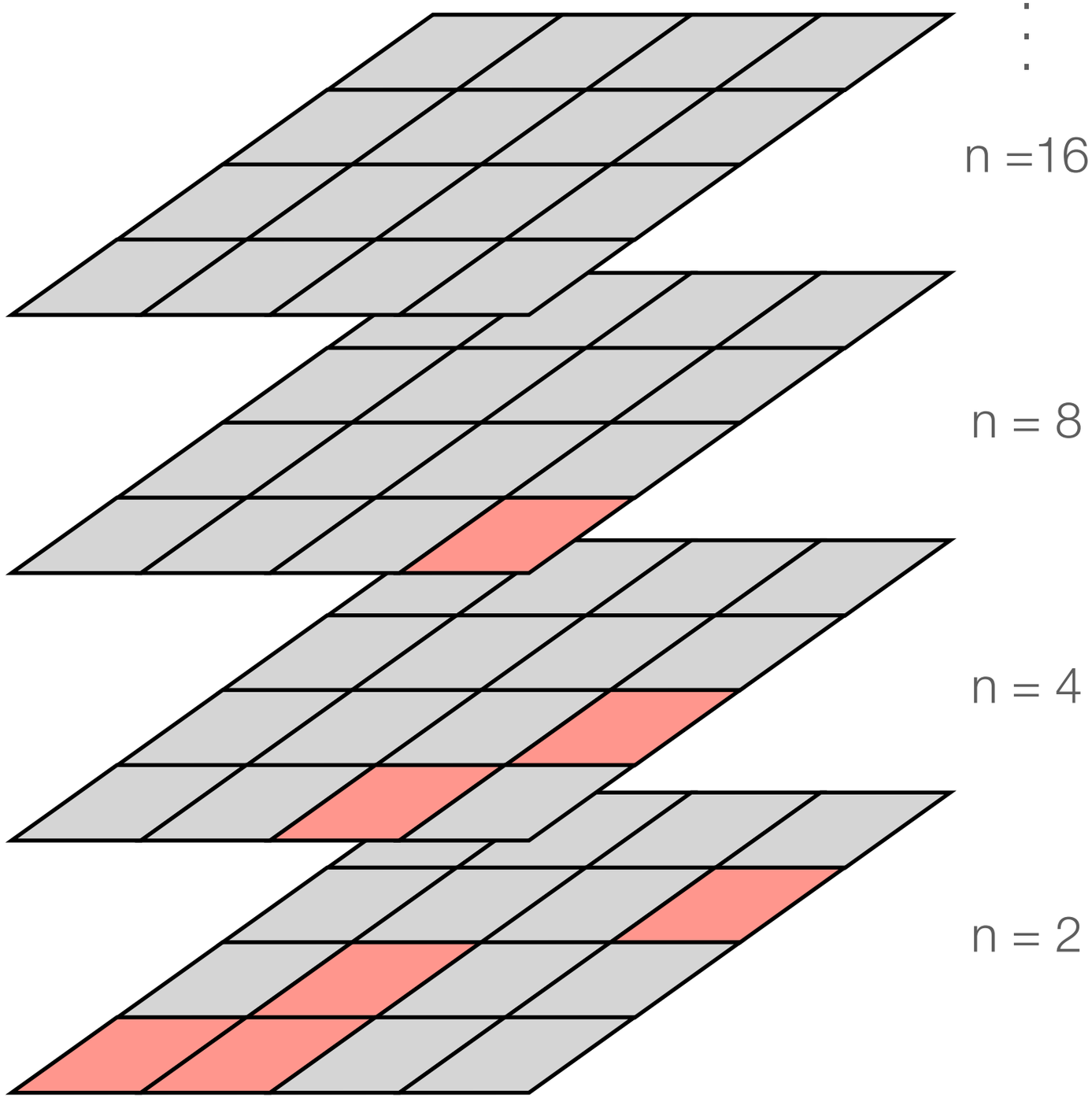
- State: Box(0,1, (16, 4, 4))
- Action: Discrete(4)

  (↑, →, ↓, ←)

State visualization:

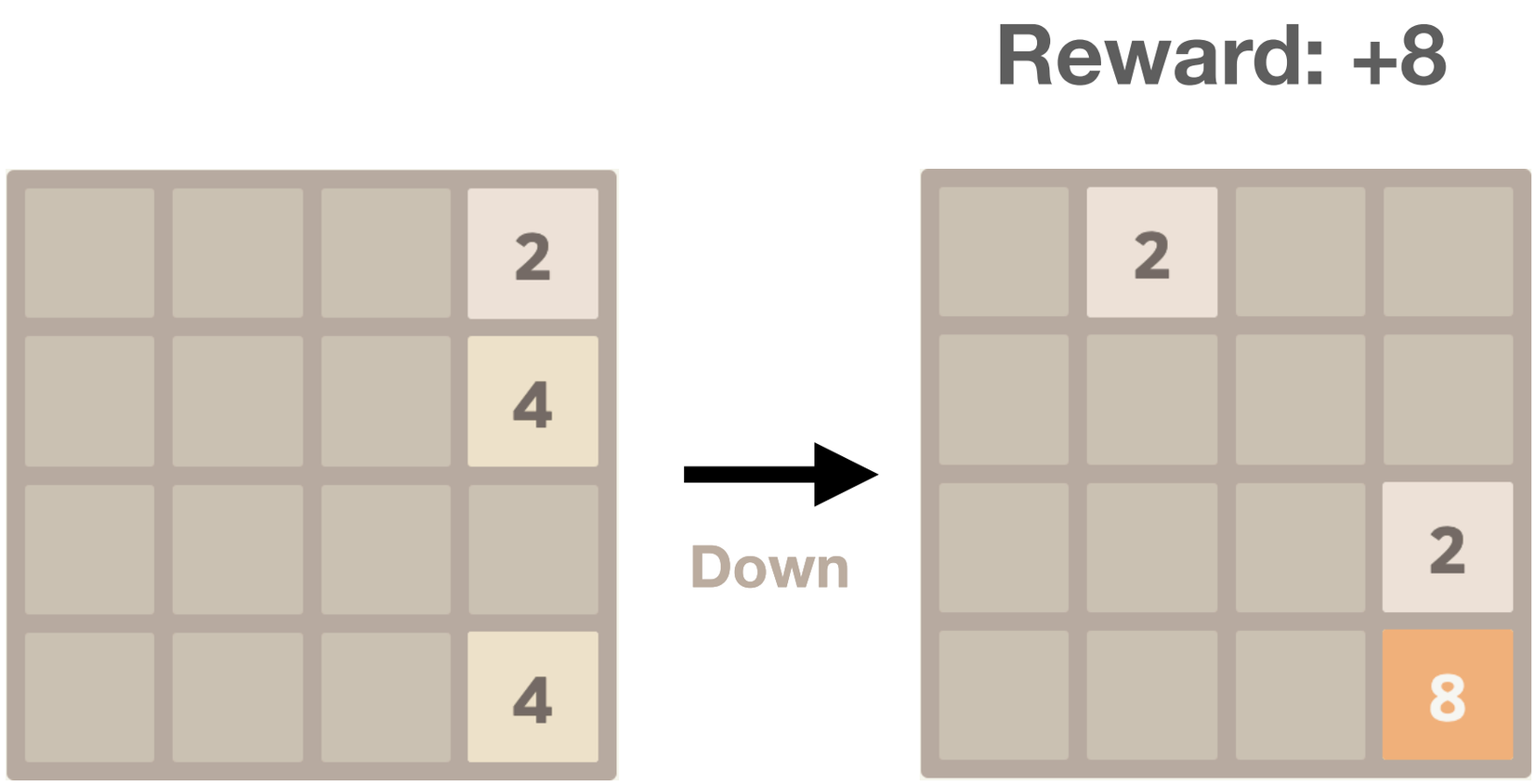


=



Env 1 - 2048 - Reward

- The default reward will be the added score for each step.
- Grades will only consider the “highest tile value”, not the reward.
- You can modify the reward during the training stage.



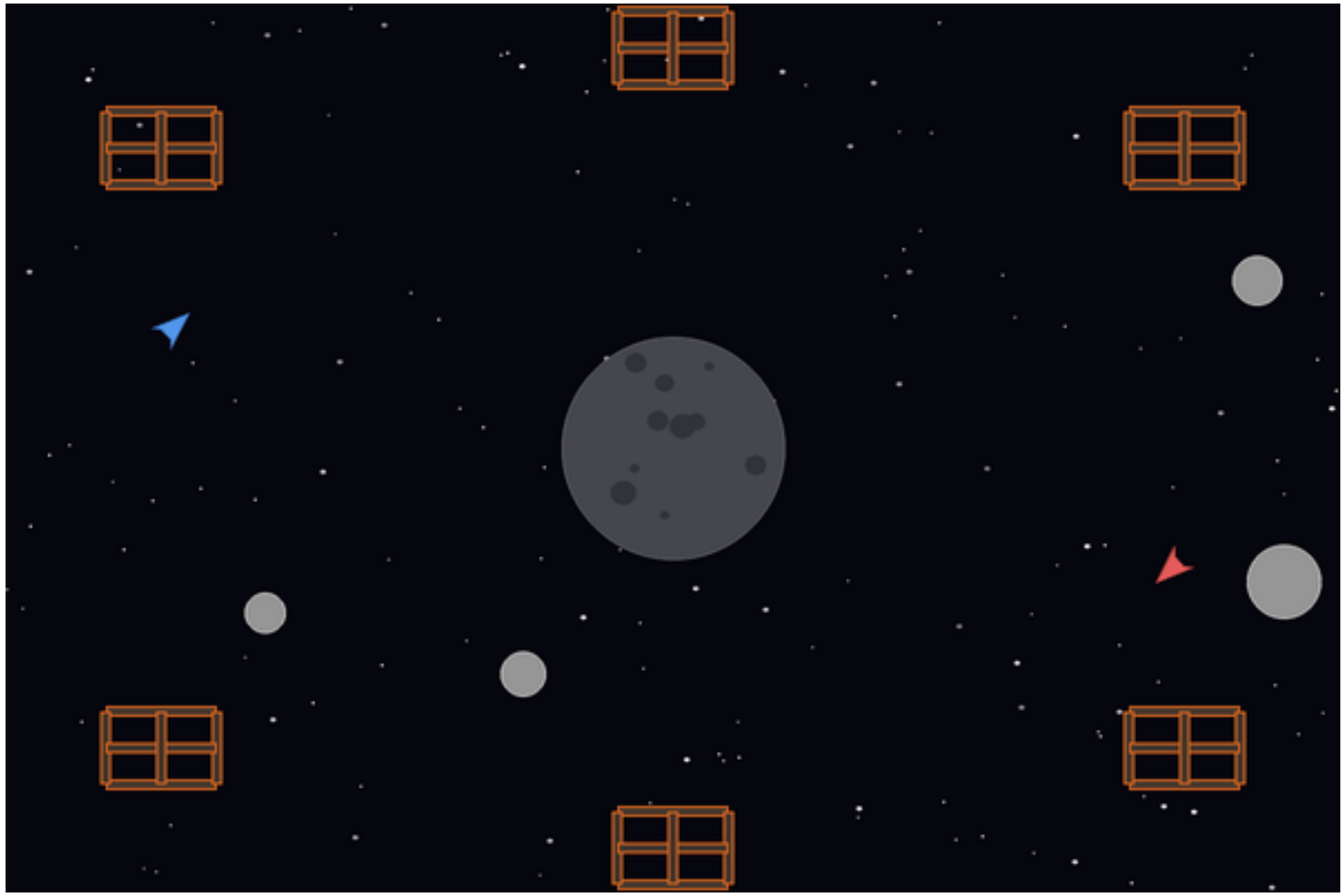
Env 1 - 2048 - Tasks

- A simulated 2048 environment will be provided, you will:
  - Train the agent with the DQN algorithm.
  - Inherit from “DQN” class to implement DDQN
  - Compare their results
  - Modify the environment if necessary (Terminal conditions, reward shaping…).
  - Work on other techniques that can help agent perform better.

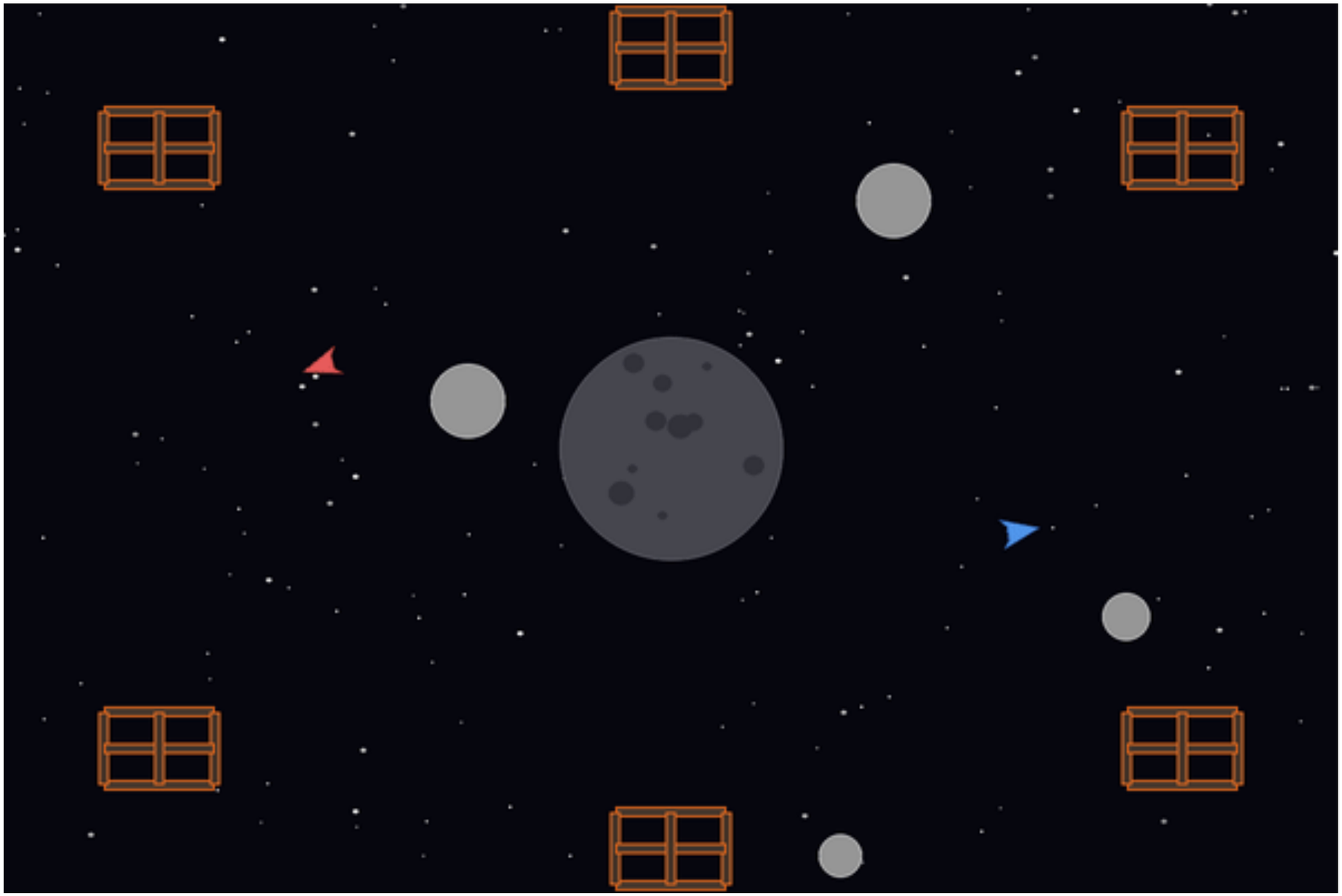
- You can submit either DQN/DDQN’s checkpoint.
- Write a report to let TA know what you have done.

Env 2 - Astro Party

Env 2 - Astro Party - Gameplay



Env 2 - Astro Party - Gameplay



Env 2 - Astro Party - Rules

- 1v1 space duel: destroy the enemy ship, then finish off its pilot.

- Rules:
  - Win: Shoot ship -> astronaut ejects -> kill the astronaut
    (With bullet/laser) (With bullet/laser/ramming)
  - Time limit: 1500 steps (~30 s). If both players are still alive, it's a draw.
  - Ammo: a 3-bullet magazine, refilled at 1 bullet/50 steps (~1 s)
  - Lasers: Destroy asteroid -> laser drop (50%). One-shot, instant beam.
  - Astronaut: can't shoot, only fly and dodge. Survive 200 steps (~4 s) -> new ship.
  - Obstacles: the planet and the cover walls block both ships and shots.
  - Spawn: random each game, mirrored across the planet, at least 500 px apart.

- Notes:
  - The map is 1200x800 pixels
  - Center is at (600, 400)
  - A lot of states are defined in pixel units (position, speed…)

Env 2 - Astro Party - State space

Overview

Indices	Block	Dims	Contents
0–13	Own ship	14	Position, velocity, heading, distances, weapon state, ejection state
14–25	Opponent (relative to self)	12	Relative position / velocity / heading, opponent status
26–37	Asteroids	12	4 slots × (dx, dy, radius)
38–43	Pickups	6	2 slots × (dx, dy, active)
44–63	Own bullets	20	4 slots × (dx, dy, vx, vy, active), nearest first
64–83	Enemy bullets	20	4 slots × (dx, dy, vx, vy, active), nearest first
	Total	84	obs[i] for i in 0..83

Env 2 - Astro Party - State - Own ship

Own ship

Index	Field	Dims	Type	Description
0–1	Position (x, y)	2	Continuous	(x – 600) / 600, (y – 400) / 400; arena centre = 0
2–3	Velocity (vx, vy)	2	Continuous	Divided by MAX_SPEED (260)
4–5	Heading (cos θ, sin θ)	2	Continuous	Nose points along (sin θ, –cos θ)
6	Dist to planet	1	Continuous	Distance to planet centre / 600
7	Dist to nearest cover	1	Continuous	Distance to nearest cover bar (田字 structures) / 600
8	Ammo fraction	1	Discrete (4 levels)	ammo / MAX_AMMO (3): 0, 1/3, 2/3, 1
9	Reload fraction	1	Continuous	Progress to next round; 1 when magazine is full
10	Has laser	1	Discrete (binary)	Holding a laser pickup (next shot fires the laser)
11	Ready	1	Discrete (binary)	Fire cooldown over (6 steps after each shot); firing also needs ammo or a laser
12	Is astronaut	1	Discrete (binary)	1 while ejected (cannot shoot)
13	Respawn fraction	1	Continuous	Respawn wait remaining, 1 → 0 while ejected; 0 otherwise

Env 2 - Astro Party - State - Opponent

Opponent (relative to self)

Index	Field	Dims	Type	Description
14–15	Relative position (dx, dy)	2	Continuous	Opponent – self: dx / 600, dy / 400
16–17	Relative velocity (dvx, dvy)	2	Continuous	Opponent – self, / MAX_SPEED (260)
18–19	Relative heading (cos, sin)	2	Continuous	Of (opponent angle – own angle)
20	Opponent alive	1	Discrete (binary)	0 once eliminated
21	Opponent is astronaut	1	Discrete (binary)	1 while ejected
22	Opponent ammo fraction	1	Discrete (4 levels)	ammo / MAX_AMMO (3)
23	Opponent reload fraction	1	Continuous	Progress to next round
24	Opponent has laser	1	Discrete (binary)	Holding a laser pickup
25	Opponent ready	1	Discrete (binary)	Fire cooldown over

Env 2 - Astro Party - State - Asteroids and pickups

Asteroids

Index (slot k = 0..3)	Field	Dims	Type	Description
26+3k, 27+3k	Relative position (dx, dy)	8	Continuous	Asteroid – self: dx / 600, dy / 400
28+3k	Radius	4	Continuous	Radius / 600

Pickups

Index (slot j = 0..1)	Field	Dims	Type	Description
38+3j, 39+3j	Relative position (dx, dy)	4	Continuous	Pickup – self: dx / 600, dy / 400
40+3j	Active flag	2	Discrete (binary)	0 when the slot has no live pickup (whole slot is 0)

Env 2 - Astro Party - State - Bullets (sorted by closeness)

Own bullets

Index (slot k = 0..3)	Field	Dims	Type	Description
44+5k, 45+5k	Relative position (dx, dy)	8	Continuous	Bullet – self: dx / 600, dy / 400
46+5k, 47+5k	Velocity (vx, vy)	8	Continuous	Divided by BULLET_SPEED (620) unit length when active
48+5k	Active flag	4	Discrete (binary)	0 for padding slots (whole slot is 0)

Enemy bullets

Index (slot k = 0..3)	Field	Dims	Type	Description
64+5k, 65+5k	Relative position (dx, dy)	8	Continuous	Bullet – self: dx / 600, dy / 400
66+5k, 67+5k	Velocity (vx, vy)	8	Continuous	Divided by BULLET_SPEED (620) unit length when active
68+5k	Active flag	4	Discrete (binary)	0 for padding slots (whole slot is 0)

Env 2 - Astro Party - Action space

Action space

Index	Field	Type	Range	Description
0	Turn rate	Continuous	[–1, 1]	× 3.5 rad/s; positive turns clockwise on screen
1	Thrust	Continuous	[–1, 1]	Accelerate along heading; reverse (< 0) at half strength
2	Shoot	Continuous	[–1, 1]	Fires if > 0 (cooldown over, ammo or laser); ignored while ejected

Env 2 - Astro Party - Tasks

- The environment and 2 TA bots will be provided, you will:
  - Try at least 2 SB3 algorithms to train your agent. (Submit the best one for grading)
  - Compare their results.
  - Modify the environment (Terminal conditions, reward shaping…).
  - Use any technique to be robust against TA bots and peers
- Write a report to let TA know what you have done.

Code Structure

Root directory

- rl_assignment_3/
  - 2048/
  - astro_party/
  - requirements.txt
  - README.md

requirements.txt

- Conda

```
conda create -n rl_assignment_3 python=3.10
conda activate rl_assignment_3
pip install -r requirements.txt
```

- venv
  - Sorry for virtual environment users. Stable Baseline 3 is hard to install using venv with pip. Do not use venv in PA3.



2048

Python Files

- train_2048.py
  - Sample code for training your model with SB3.
  - Modify my_config to set hyper parameters and configurations.
- eval_2048.py
  - Load saved model and run 100 rollouts for evaluation.

- Note: You are encouraged to modify “train_2048.py” and “train_2048_env.py” as much as you like. Still, do keep in mind that the model will be put in our setting for evaluation.

Other folders

- envs/
  - eval_2048_env.py
    - Environment that will be used for evaluating agent's performance.
    - TA will use "eval_2048.py", "eval_2048_env.py" for evaluation, don't modify them.
  - train_2048_env.py
    - Environment used for training your agent.
    - Feel free to modify any parts of this file to help agent learn better.
- models/
  - sample_model.zip
    - A loadable dummy model for evaluation reference.

astro_party

Python Files

- train_astro_party.py
  - Sample code for training your model with SB3.
  - Modify my_config to set hyper parameters and configurations.
- eval_astro_party.py
  - Load saved model and run 100 rollouts for evaluation.

- Note: You are encouraged to modify “train_astro_party.py” and “train_astro_party_env.py” as much as you like. Still, do keep in mind that the model will be put in our setting for evaluation.

Python Files

- envs/
  - eval_astro_party_env.py
    - Environment that will be used for evaluating agent's performance.
    - TA will use "eval_astro_party.py" and "eval_astro_party_env.py" for evaluation, so don't modify them.
  - train_astro_party_env.py
    - Environment used for training your agent.
    - Feel free to modify any parts of this file to help agent learn better. (Rewards etc.)
  - game.py
    - Game logic for building env
    - Do not modify
  - render.py
    - Graphics of the game
    - Do not modify

Other folders

- models/
  - rusher_bot.py
    - Medium rule-based TA bot, used as baseline
  - sniper_bot.py
    - Strong rule-based TA bot, used as baseline
  - sample_model.zip
    - A loadable dummy model for evaluation reference.

Grading

Grading

- 2048 (30%)
  - Simple baseline: Reach 128 for at least 1 rollout / 100 rollouts. (Public: 5%, Private: 5%)
  - Medium baseline: Reach 256 for at least 1 rollout / 100 rollouts. (Public: 5%, Private: 5%)
  - Strong baseline: Reach 512 for at least 1 rollout / 100 rollouts. (Public: 5%, Private: 5%)
- Astro Party (40%)
  - Medium baseline: reach win rate of 90% against rusher_bot (Public: 5%, Private: 5%)
  - Strong baseline: reach win rate of 85% against sniper_bot (Public: 5%, Private: 5%)
  - Round robin: rank win rate among peers, score is linearly decreased w.r.t. rank, i.e., first place gets 20%, last place gets 0%, ties share the better rank (Private: 20%)
- Report (30%)
  - See “Report” slides for details

Criteria - 2048

- Test cases:
  - We will run your model on seeded environments and check the best tile value (“Highest”) for baseline evaluations.
  - Public test cases: 100 rollouts, using seed 0 to 99.
  - Private test cases: 100 rollouts, using another 100 seeds selected by TA.
  - Public and private will be tested separately.
  - Run time for each case is limited to **10 minutes** to prevent infinite loops.

- Only 1 model will be loaded and tested for both test cases.
- For grading, we will use “eval_2048.py” and “eval_2048_env.py” to evaluate your performance, please make sure that **“eval_2048.py” can load your model** without modifying it.
- Model should be trained with algorithms from SB3.
- Model parameters count shouldn’t exceed 5 million. (excluding target network)
- Please make sure that we can briefly reproduce your results.

Criteria - Astro Party

- Test cases:
  - We will run your model on seeded environments (randomly spawned) and calculate win rate.
  - Public test cases: 100 rollouts, using seed 0 to 99.
  - Private test cases: 100 rollouts, using another 100 seeds selected by TA.
  - Public and private will be tested separately.
  - Episodes that reach 1500 MDP steps (from one agent) count as a draw.

- Round robin (10 games vs. each peer, seeds chosen by TA)
  - Whoever wins more of the 10 games wins the match.
  - Drawn games, tied matches are not wins.
  - Players are ranked by match win rate. Score goes linearly from 20% (1st) to 0% (last).

Criteria - Astro Party

- Only 1 model will be loaded and tested for both test cases.
- You may duel against TA's bots to train your RL agent.
- For grading, we will use "eval_astro_party.py" and "eval_astro_party_env.py" to evaluate your performance, please make sure that **"eval_astro_party.py" can load your model** without modifying it.
- Model should be trained with algorithms from SB3.
- Model parameters count shouldn't exceed 2 million. (excluding target network)
- Please make sure that we can briefly reproduce your results.
- You can share checkpoints with peers as "adversary ship". However, we will check your model weights correlation. If your checkpoint is fine-tuned from your peer's model or straight up plagiarism, we will find out and both of you will get a score of 0.

Report

Report

- 2048 (12%)
  - Q1. Given π = argmax Q, implement Monte Carlo evaluation of Q^π(s,a). State how you handle tile-spawn randomness, the number of rollouts, and confidence intervals. (3%)
  - Q2. For DQN and DDQN, plot predicted Q against MC-estimated Q^π on states visited by π. Is there a discrepancy? Explain the difference between the two algorithms. Does a small discrepancy imply a good policy? (5%)
  - Q3. Describe how you trained your best model (architecture, reward, tricks). (4%)
- Astro Party (18%)
  - Q4. Thoroughly describe your pipeline for best performance (Be creative! Any technique is worth sharing!) (9%)
  - Q5. Try at least 2 RL algorithms from SB3, explain why one works better than the other. State at least one hypothesis and do clean experiments to justify/falsify them. (9%)
- Use Overleaf template and save in PDF format.
  - Overleaf template
- LLM-generated report will get very low score if detected.

Policy

Policy

Package

- You can use any Python standard library (e.g., heap, queue…).
- System level packages are prohibited (e.g., sys, os, shutil…), importing any of them will result in 0 score.

Collaboration

- Discussions are encouraged.

Plagiarism & cheating

- All assignment submissions will be subject to duplication checking (e.g., MOSS).
- Cheater will receive an **F** grade for this course.

LLM policy

- LLM for code writing is not prohibited. However, we will not accept using LLM as an excuse for plagiarism.
- Write your own report, LLM-generated reports will get very low scores.

Grade appeal

- Assignment grades are considered finalized two weeks after release.

Submission

Submission

- Submit on NTU COOL with following zip file structure:
  - train_2048.py, train_2048_env.py: Your implementation for 2048.
  - model_2048.zip: Saved 2048 model that can be loaded with eval_2048.py.
  - train_astro_party.py, train_astro_party_env.py: Your implementation for astro party.
  - model_astro_party.zip: Saved astro_party model that can be loaded with eval_astro_party.py.
  - Get rid of pycache, DS_Store, etc.
  - Student ID with lower case
  - 10 points reduction for wrong file format
  - 0 score for importing system level packages





- Deadline 2026/11/04 Wed 11:59pm
- No late submission is allowed

Contact

Questions?

- General questions
  - Use channel **#assignment** in slack as first option
  - Reply in thread to avoid spamming other people
- Personal questions
  - DM us on Slack: **TA 張冲和 r15942081**



TA 張冲和 r15942081